laborbericht.

Biosignalverarbeitung Laborbericht (BA-MGST-B-3-BB-BS-ILV)

Aufbau der Hardware & Datenakquise

Labor 1 – Gruppe X

Medizin- Sport- und Gesundheitstechnik

3. Semester

Lehrveranstaltungsleiter: Aitor Morillo

Jahrgang: MGST-25

Verfasser: Matthias Zoller, Laurenz Keller, Matteo Harrasser

27. 10. 2026

inhaltsverzeichnis.

1 Einleitung **3**

2 Aufbau des Messsystems (Aufgabe 1) **3**

3 Vorausgehende Arbeiten und Berechnungen **4**

3.1 Umrechnung eines ADC-Wertes in eine Temperatur (Aufgabe 2) 4

3.2 Achsenzuordnung und Umrechnung in physikalische Größen (Aufgabe 3) 4

4 Durchführung und Ergebnisse **4**

4.1 Auf- und Abbewegung mit dem Originalcode (Aufgabe 4) 4

4.2 Anpassung der Sensorkonfiguration (Aufgabe 5) 5

4.3 Zeitstempel und statische Messung in sechs Orientierungen (Aufgabe 6) 6

4.4 Filterung der Daten (Aufgabe 7) 7

4.5 Bestimmung der Abtastfrequenz (Aufgabe 8) 7

4.6 Mobiles Messsystem (Aufgabe 9) 8

5 Zusammenfassung **8**

A Anhang: Abgegebene Dateien (Aufgabe 10) **9**

abbildungsverzeichnis.

1 Aufbau des stationären Messsystems. Schwarz: Datenpfade, rot gestrichelt: Spannungsversorgung. 3

2 Auf- und Abbewegung mit `Lab1Code1` (ODR 12,5 Hz), Ausschnitt 4 s. 5

3 Vergleich je 4 s: oben ODR 12,5 Hz (`Lab1Code1`), unten ODR 100 Hz (modifizierter Code). 6

4 Sensor in sechs Lagen à ca. 10 s; rot: Betrag $|a| = \sqrt{x^2 + y^2 + z^2}$. 6

5 Ungefilterte (hell) und tiefpassgefilterte Daten ($f_g = 5$ Hz), Ausschnitt 5–15 s. . . 7

6 Histogramme der Abtastintervalle Δt der drei Messungen. 8

tabellenverzeichnis.

1 Mögliche Werte des Parameters `acc_odr` im Register `ACC_CONFIG1` (0x1A) des BMA400. 5

2 Mittelwerte der Beschleunigung in den sechs Lagen (Werte in g). 7

3 Abtastintervalle und Abtastfrequenzen der Messungen. 8

1 einleitung

Ziel des ersten Labors ist der Aufbau eines Messsystems zur Erfassung von Beschleunigungsdaten sowie deren Verarbeitung in Python. Dazu wird der 3-Achsen-Beschleunigungssensor BMA400 über I^{2}C an einen Arduino Uno angebunden, die Daten über die serielle Schnittstelle an einen PC übertragen und anschließend in Python eingelesen, gefiltert und dargestellt. Untersucht werden insbesondere der Einfluss der Datenrate (ODR) des Sensors auf die Qualität der Messdaten, die tatsächlich erreichte Abtastfrequenz sowie der Unterschied zwischen stationärem (USB) und mobilem (DataLogger, Batterie) Messsystem.

Der Python-Code `MainCode1.ipynb` bearbeitet die Aufgaben 4 (a), 5, 6 (a), 7, 8, 9 und 10 und erzeugt alle Abbildungen dieses Berichts.

2 aufbau des messsystems (aufgabe 1)

Abbildung 1 zeigt das stationäre Messsystem mit allen Komponenten, Bussen und Signalpfaden.

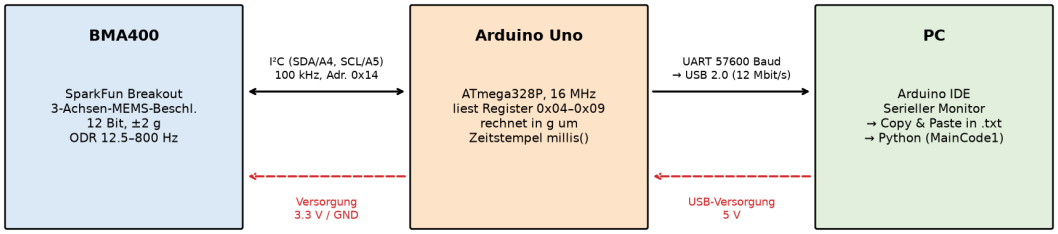


Abbildung 1: Aufbau des stationären Messsystems. Schwarz: Datenpfade, rot gestrichelt: Spannungsversorgung.

KOMPONENTEN

BMA400 (SparkFun Breakout): Digitaler 3-Achsen-MEMS-Beschleunigungssensor mit 12 Bit Auflösung. Messbereich (±2 bis ±16 g) und Datenrate (ODR, 12,5–800 Hz) sind einstellbar; die Messwerte stehen in den Registern 0x04–0x09.

Arduino Uno (ATmega328P, 16 MHz): Konfiguriert den Sensor, liest die sechs Datenregister aus, rechnet die Rohwerte in g um und hängt den Zeitstempel `millis()` an. Er versorgt den Sensor mit 3,3 V.

PC (Arduino IDE): Der serielle Monitor zeigt die Messwerte zeilenweise (`X Y Z Time`) an; sie werden per Copy & Paste in eine `.txt`-Datei übernommen.

Python (`MainCode1.ipynb`): Liest die Textdatei mit `pandas` ein, filtert mit `SciPy` und stellt die Daten mit `matplotlib` dar.

BUSSE UND SIGNALPFADE

I^{2}C (100 kHz): Zweidraht-Bus (SDA = A4, SCL = A5). Der Arduino ist Master, der Sensor Slave mit der Adresse 0x14.

UART / USB 2.0: Der Arduino sendet seriell mit 57 600 Baud; ein USB-Seriell-Wandler auf dem Board setzt dies auf USB 2.0 Full Speed (12 Mbit/s) um.

Datenpfad PC → Python: Übergabe der Messdaten als Textdatei.

Spannungsversorgung: 5 V über USB für den Arduino, 3,3 V und GND vom Arduino zum Sensor.

3 vorausgehende arbeiten und berechnungen

3.1 UMRECHNUNG EINES ADC-WERTES IN EINE TEMPERATUR (AUFGABE 2)

Gegeben sind ein Temperatursensor mit $U = 0 \ldots 3{,}3\,\mathrm{V}$ für $\vartheta = 0 \ldots 40\,^\circ\mathrm{C}$ (linear) und ein ADC mit $n = 10\,\mathrm{Bit}$ $(0 \ldots 1023)$ und Referenzspannung $U_{\mathrm{ref}} = 5\,\mathrm{V}$. Der ADC liefert $D = 520$.

Schritt 1 – ADC-Wert in Spannung:

$$U = \frac{D}{2^n - 1} \cdot U_{\mathrm{ref}} = \frac{520}{1023} \cdot 5\,\mathrm{V} \approx 2{,}542\,\mathrm{V} \tag{1}$$

Schritt 2 – Spannung in Temperatur:

$$\vartheta = \frac{U}{3{,}3\,\mathrm{V}} \cdot 40\,^\circ\mathrm{C} = \frac{2{,}542\,\mathrm{V}}{3{,}3\,\mathrm{V}} \cdot 40\,^\circ\mathrm{C} \approx \mathbf{30{,}8\,^\circ C} \tag{2}$$

Der Sensor misst also ca. 30,8 °C, was für eine Messung auf der Haut plausibel ist (die Hauttemperatur liegt unter der Körperkerntemperatur von ca. 37 °C). Da der Sensor höchstens 3,3 V liefert, werden nur ADC-Werte bis $\frac{3{,}3}{5} \cdot 1023 \approx 675$ genutzt; die Auflösung beträgt damit $40\,^\circ\mathrm{C}/675 \approx 0{,}06\,^\circ\mathrm{C}$ pro Stufe statt der möglichen 0,04 °C.

3.2 ACHSENZUORDNUNG UND UMRECHNUNG IN PHYSIKALISCHE GRÖßEN (AUFGABE 3)

Achsenzuordnung: `Lab1Code1` gibt pro Zeile `X Y Z Time` aus; die Spalten entsprechen der Reihenfolge der Datenregister (0x04/0x05 = x, 0x06/0x07 = y, 0x08/0x09 = z) und damit den auf der Platine aufgedruckten Achsen.

Test: Der Sensor wird so hingelegt, dass eine Achse senkrecht nach oben zeigt. Diese Achse zeigt dann ca. $+1\,g$, die anderen ca. $0\,g$. Flach auf dem Tisch (z nach oben) wurde z. B. `7 -91 1002` gemessen, also $z \approx 1\,g$. Die systematische Prüfung aller sechs Lagen erfolgt in Aufgabe 6.

Einheit: Ausgegeben werden Rohwerte in LSB (Digits): pro Achse zwei Register (LSB, MSB), die zusammen einen 12-Bit-Wert im Zweierkomplement $(-2048 \ldots +2047)$ ergeben: `raw = ((msb & 0x0F) « 8) | lsb`, danach bei `raw > 2047` → `raw -= 4096`.

Umrechnung: Die Empfindlichkeit ergibt sich laut Datenblatt [1] [**TODO:** Seite/Tabelle der Sensitivity eintragen] zu $S = 2048/\text{Messbereich}$, also $S = 1024\,\mathrm{LSB}/g$ bei $\pm 2\,g$:

$$a\,[g] = \frac{\mathrm{raw}}{1024}, \qquad a\,[\mathrm{m/s^2}] = \frac{\mathrm{raw}}{1024} \cdot 9{,}81\,\mathrm{m/s^2} \tag{3}$$

Beispiel: $1002/1024 = 0{,}98\,g = 9{,}60\,\mathrm{m/s^2}$.

4 durchführung und ergebnisse

4.1 AUF- UND ABBEWEGUNG MIT DEM ORIGINALCODE (AUFGABE 4)

Der Sensor wurde in der geschlossenen Hand gehalten und ca. einmal pro Sekunde schnell auf und ab bewegt (ohne Rotation). Die Daten wurden mit `Lab1Code1` (ODR 12,5 Hz, $\pm 2\,g$) aufgezeichnet und in eine Textdatei kopiert.

(a) Abbildung 2 zeigt 4 s der Messung, umgerechnet in g.

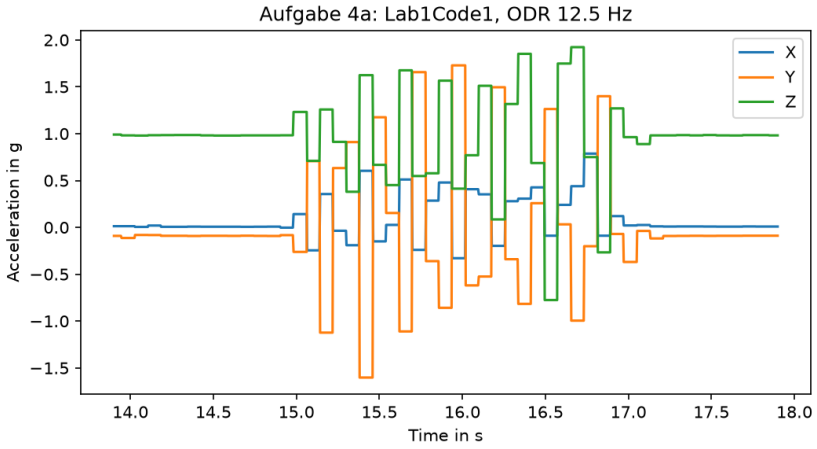


Abbildung 2: Auf- und Abbewegung mit `Lab1Code1` (ODR 12,5 Hz), Ausschnitt 4 s.

(b) Plateaus der Peaks: Jeder Wert bleibt ca. 80 ms konstant, wodurch die treppenförmigen Plateaus entstehen. Der Sensor liefert neue Werte nur mit der ODR von 12,5 Hz (alle 80 ms), die `loop()` gibt ohne `delay` jedoch ca. alle 3 ms (≈ 313 Hz, vgl. Tab. 3) eine Zeile aus. Bis ein neuer Wert im Datenregister steht, wird also immer wieder derselbe Wert gelesen (ca. 25 identische Zeilen pro Messwert). **Lösung:** Die ODR im Register `ACC_CONFIG1` erhöhen (Aufgabe 5) und die Ausgaberate mit einem passenden `delay` an die ODR anpassen. Zusätzlich liegen die Peaks in z (inkl. Erdbeschleunigung) mit ca. 1,9 g nahe an der Messbereichsgrenze von $\pm 2\,g$; für stärkere Bewegungen sollte daher auch der Messbereich vergrößert werden, um Sättigung zu vermeiden.

(c) Abweichung von der tatsächlichen Bewegung: Bei 12,5 Hz liegt die Nyquist-Frequenz bei 6,25 Hz. Die schnelle Bewegung enthält höhere Frequenzanteile, sodass *Aliasing* entsteht: Pro Bewegung gibt es nur wenige Messpunkte, das Signal springt (v. a. in y) fast bei jedem Sample das Vorzeichen, statt den Bewegungsverlauf zu zeigen. Außerdem misst der Sensor immer die Erdbeschleunigung mit; dreht sich die Hand leicht, verteilt sich diese auf mehrere Achsen.

4.2 ANPASSUNG DER SENSORKONFIGURATION (AUFGABE 5)

Die Datenrate wird laut Datenblatt BMA400 [1] (Rev. 2.3, **S. 64**) im Register `ACC_CONFIG1` (0x1A) über den Parameter `acc_odr` (Bits 3:0) eingestellt. Die möglichen Werte zeigt Tab. 1.

Tabelle 1: Mögliche Werte des Parameters `acc_odr` im Register `ACC_CONFIG1` (0x1A) des BMA400.

`acc_odr`	0x00–0x04	0x05	0x06	0x07	**0x08**	0x09	0x0A	0x0B	0x0C–0x0F
ODR / Hz	reserviert	12,5	25	50	**100**	200	400	800	reserviert

Im Code (`Lab1Code1_modifiziert.ino`) wurden folgende Zeilen geändert:

```
accel.setODR(BMA400_ODR_100HZ); // ACC_CONFIG1 (0x1A), acc_odr = 0x08
...
delay(10); // Ausgaberate an ODR anpassen: 1/100 Hz = 10 ms
```

Begründung: Handbewegungen liegen deutlich unter 50 Hz (Nyquist-Frequenz bei 100 Hz) und werden damit ohne Aliasing erfasst. Höhere Raten sind nicht sinnvoll: Eine Ausgabezeile (`-0.1234\t` ×3 + Zeitstempel + Zeilenumbruch) umfasst ca. 30 Zeichen ≈ 300 Bit, bei 57 600 Baud sind also nur ca. 190 Zeilen/s übertragbar – 200 Hz und mehr würden die serielle Schnittstelle überlasten.

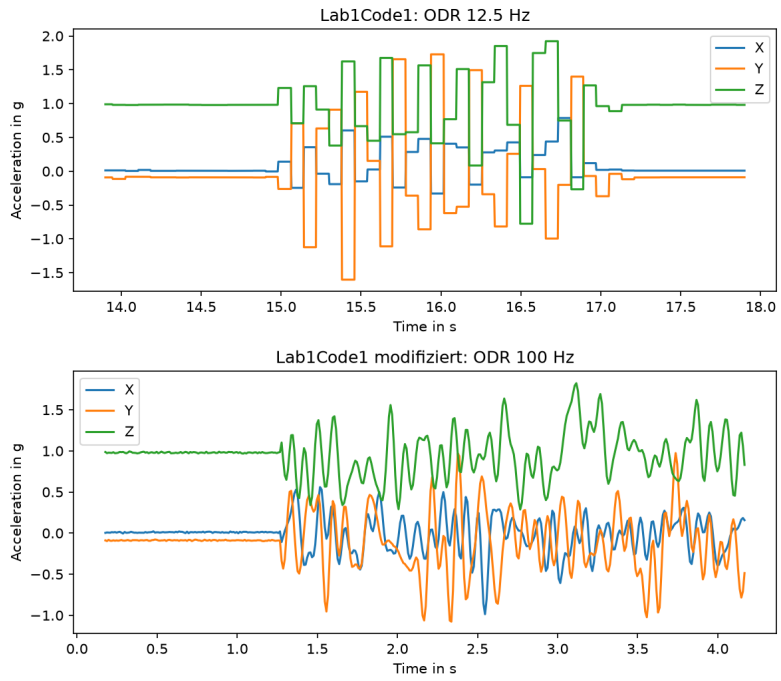


Abbildung 3: Vergleich je 4 s: oben ODR 12,5 Hz (`Lab1Code1`), unten ODR 100 Hz (modifizierter Code).

Vergleich (Abb. 3): Gleich sind der Ruhepegel ($z \approx 1\,g$, x und $y \approx 0$) und die Größenordnung der Ausschläge (bis ca. $1{,}8\,g$). Bei 100 Hz gibt es jedoch keine Treppenstufen mehr: Die Kurve ist glatt und die einzelnen Auf-ab-Bewegungen sind als Schwingung zu erkennen, während das Signal bei 12,5 Hz durch das Aliasing zerhackt ist. Ursache ist ausschließlich die höhere Abtastrate; Sensor und Messbereich sind gleich geblieben.

4.3 ZEITSTEMPEL UND STATISCHE MESSUNG IN SECHS ORIENTIERUNGEN (AUFGABE 6)

Jede Zeile erhält direkt auf dem Arduino den Zeitstempel `millis()` (4. Spalte), unabhängig von den Zeitstempeln des seriellen Monitors. Daraus wird in Abschnitt 4.5 die mittlere Abtastfrequenz berechnet. Der Sensor wurde nacheinander in alle sechs Lagen gebracht und jeweils ca. 10 s ruhig gehalten.

(a) Abbildung 4 zeigt die gesamte Messung.

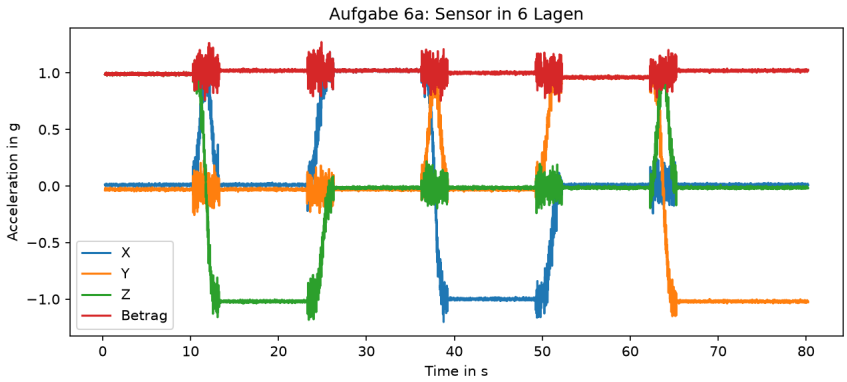


Abbildung 4: Sensor in sechs Lagen à ca. 10 s; rot: Betrag $|a| = \sqrt{x^2 + y^2 + z^2}$.

(b) Plausibilität: Tabelle 2 zeigt die Mittelwerte in den Ruhephasen.

Tabelle 2: Mittelwerte der Beschleunigung in den sechs Lagen (Werte in g).

Lage	x	y	z	$\lvert a\rvert$
z oben	0,01	−0,03	**0,99**	0,99
z unten	0,01	−0,03	**−1,02**	1,02
x oben	**1,02**	−0,03	−0,02	1,02
x unten	**−1,00**	−0,03	−0,02	1,00
y oben	0,01	**0,96**	−0,02	0,96
y unten	0,01	**−1,02**	−0,02	1,02

Die Daten ergeben Sinn: Die senkrecht stehende Achse zeigt immer ca. $\pm 1\,g$, die anderen ca. $0\,g$, und der Betrag liegt stets bei ca. $1\,g$. Zeigt eine Achse nach oben, ist der Wert positiv, weil der Sensor die Gegenkraft der Unterlage misst. Die kleinen Abweichungen (bis $0{,}04\,g$) entstehen durch Offset-Fehler des Sensors und dadurch, dass er von Hand nicht exakt gerade gehalten wurde.

4.4 FILTERUNG DER DATEN (AUFGABE 7)

Die Daten aus Aufgabe 6 wurden mit einem Butterworth-Tiefpass 4. Ordnung mit der Grenzfrequenz $f_g = 5\,\text{Hz}$ gefiltert. Dabei wurde `filtfilt()` aus `scipy.signal` [3] verwendet, das den Filter vorwärts und rückwärts anwendet und so keine Phasenverschiebung erzeugt (Nullphasenfilter).

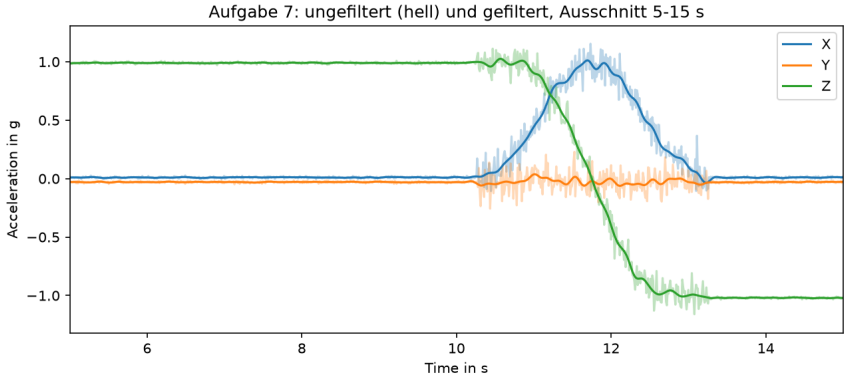


Abbildung 5: Ungefilterte (hell) und tiefpassgefilterte Daten ($f_g = 5\,\text{Hz}$), Ausschnitt 5–15 s.

Effekt und Begründung: Der Filter entfernt das Sensorrauschen und das Zittern der Hand (in Ruhe sinkt die Standardabweichung von z von 6,0 auf $1{,}6\,\text{m}g$), während die Lagewechsel erhalten bleiben und durch `filtfilt` nicht verschoben werden (Abb. 5). Ein Tiefpass wurde gewählt, weil sich die Lage nur langsam ändert, das Rauschen aber hochfrequent ist.

4.5 BESTIMMUNG DER ABTASTFREQUENZ (AUFGABE 8)

Aus den `millis()`-Zeitstempeln wurden die Abtastintervalle $\Delta t_i = t_{i+1} - t_i$ und daraus

$$\bar{f}_s = \frac{1}{\overline{\Delta t}}, \qquad \sigma_{f_s} \approx \frac{\sigma_{\Delta t}}{\overline{\Delta t}^{\,2}} \tag{4}$$

bestimmt (Fehlerfortpflanzung). Abbildung 6 zeigt die Verteilung der Intervalle, Tab. 3 die Ergebnisse.

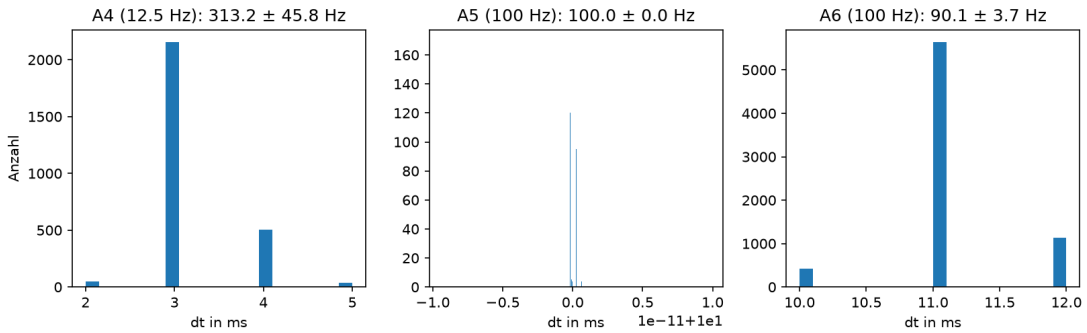


Abbildung 6: Histogramme der Abtastintervalle Δt der drei Messungen.

Tabelle 3: Abtastintervalle und Abtastfrequenzen der Messungen.

Messung	$\overline{\Delta t}$ / ms	$\sigma_{\Delta t}$ / ms	$\bar{f}_s$ / Hz
A4: `Lab1Code1` (ODR 12,5 Hz)	3,19	0,47	313 ± 46
A5: modifiziert (ODR 100 Hz)	10,00	0,00	$100{,}0 \pm 0{,}0$
A6: 6 Lagen (ODR 100 Hz)	11,10	0,46	$90{,}1 \pm 3{,}7$
A9: mobiles System (`Lab1Code2`)	**[TODO: ?]**	**[TODO: ?]**	**[TODO: ?]**

Bei `Lab1Code1` gibt die Schleife ohne `delay` mit ca. 313 Hz aus, neue Sensorwerte kommen aber nur mit 12,5 Hz. Mit dem geänderten Code passen Ausgabe und ODR zusammen und die Abtastfrequenz liegt bei ca. 100 Hz. Sie liegt in Messung A6 etwas darunter (90 Hz), weil zu `delay(10)` noch die Zeit für das Auslesen über I^2C und `Serial.print` hinzukommt ($\Delta t \approx 11$ ms). Da `millis()` nur auf 1 ms genau auflöst, wäre `micros()` für eine genauere Bestimmung besser geeignet. [**TODO:** Messung A5 prüfen: Exakt 10,00 ms bei jedem Sample ($\sigma = 0$) passt nicht zu A6 mit demselben Code (11,1 ms). Wurde A5 mit anderem Code/Timing aufgenommen? Ggf. erklären oder neu messen.]

4.6 MOBILES MESSSYSTEM (AUFGABE 9)

[**TODO:** Messung mit DataLogger + Batterie und `Lab1Code2` (ca. 10 s) durchführen, Ergebnis in Tab. 3 eintragen und Text an die Messwerte anpassen.]

Für diese Messung wurde das mobile Messsystem mit DataLogger und Batterie verwendet. Mit `Lab1Code2` wurden ca. 10 s aufgezeichnet und die Abtastfrequenz wie in Abschnitt 4.5 bestimmt (Tab. 3).

Diskussion (Entwurf): Die Abtastfrequenz des mobilen Systems unterscheidet sich vom stationären Wert und streut stärker. Ursache ist, dass die Daten nicht mehr nur seriell an den PC ausgegeben, sondern auf eine SD-Karte geschrieben werden. Das Schreiben erfolgt blockweise und dauert unregelmäßig mehrere Millisekunden, in denen keine neuen Werte gelesen werden. Zudem unterscheiden sich der verwendete Code (`Lab1Code2`, z. B. Datenrate, Ausgabeformat, `delay`), die Übertragungsgeschwindigkeit zum Logger und ggf. die Taktquelle. Die eingestellte ODR stellt daher nur eine Obergrenze dar; die tatsächliche Abtastfrequenz wird von der langsamsten Stelle der Signalkette bestimmt.

5 zusammenfassung

Das Messsystem aus BMA400, Arduino Uno und PC wurde aufgebaut und in Betrieb genommen. Mit der Standardkonfiguration (ODR 12,5 Hz) bildeten die Daten die schnelle Bewegung wegen zu geringer Abtastrate (Aliasing) und mehrfach ausgegebener Werte nicht korrekt ab. Durch Erhöhen der ODR auf 100 Hz im Register `ACC_CONFIG1` und Anpassen der Ausgaberate

wurde die Bewegung korrekt erfasst. Die statische Messung bestätigte in allen sechs Lagen eine Beschleunigung von ca. 1 g (Abweichung $\leq 0{,}04\,g$). Ein Butterworth-Tiefpass (5 Hz, `filtfilt`) reduzierte das Rauschen deutlich ohne Phasenverschiebung. Die tatsächliche Abtastfrequenz betrug ca. 90–100 Hz (stationär) bzw. [**TODO:** ?] Hz (mobil).

literatur.

[1] Bosch Sensortec, "BMA400 – Ultra-low power triaxial accelerometer," Datenblatt, Rev. 2.3. [**TODO:** Dokumentnummer/Link eintragen.]

[2] SparkFun Electronics, "SparkFun Triple Axis Accelerometer Breakout – BMA400 (Qwiic), Hookup Guide," online. [**TODO:** URL und Abrufdatum]

[3] P. Virtanen *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.

A anhang: abgegebene dateien (aufgabe 10)

Die Abgabe `Lab1_Gruppe_X.zip` enthält:

- `Bericht_1_Gruppe_X.pdf` – dieser Bericht,
- `Lab1Code1_modifiziert.ino` – modifizierter Arduino-Code (Aufgabe 5),
- `MainCode1.ipynb` – Python-Auswertung (Aufgaben 4a, 5, 6a, 7, 8, 9); läuft komplett durch und speichert alle Abbildungen in `figures/`,
- Datensätze in `data/`: `IMU_A4_Lab1Code1_roh.csv`, `IMU_A5_Lab1Code1mod_100Hz.csv`, `IMU_A6_Lagen.csv`, [**TODO:** Datensatz Aufgabe 9].